

- 你真的需要一个新 **type** 吗？如果只是定义新的 **derived class** 以便为既有的 **class** 添加机能，那么说不定单纯定义一或多个 **non-member 函数** 或 **templates**，更能够达到目标。

这些问题不容易回答，所以定义出高效的 **classes** 是一种挑战。然而如果能够设计出至少像 C++ 内置类型一样好的用户自定义（**user-defined**）**classes**，一切汗水便都值得。

请记住

- **Class** 的设计就是 **type** 的设计。在定义一个新 **type** 之前，请确定你已经考虑过本条款覆盖的所有讨论主题。

条款 20: 宁以 **pass-by-reference-to-const** 替换 **pass-by-value**

Prefer **pass-by-reference-to-const** to **pass-by-value**.

缺省情况下 C++ 以 **by value** 方式（一个继承自 C 的方式）传递对象至（或来自）函数。除非你另外指定，否则函数参数都是以实际实参的复件（副本）为初值，而调用端所获得的亦是函数返回值的一个复件。这些复件（副本）系由对象的 **copy** 构造函数产出，这可能使得 **pass-by-value** 成为昂贵的（费时的）操作。考虑以下 **class** 继承体系：

```
class Person {
public:
    Person();                      //为求简化，省略参数
    virtual ~Person();            //条款 7 告诉你为什么它是 virtual
    ...
private:
    std::string name;
    std::string address;
};

class Student: public Person {
public:
    Student();                    //再次省略参数
    ~Student();
    ...
private:
    std::string schoolName;
    std::string schoolAddress;
};
```

现在考虑以下代码，其中调用函数 `validateStudent`，后者需要一个 `Student` 实参 (*by value*) 并返回它是否有效：

```
bool validateStudent(Student s);           //函数以 by value 方式接受学生
Student plato;                             //柏拉图,苏格拉底的学生
bool platoIsOK = validateStudent(plato);   //调用函数
```

当上述函数被调用时，发生什么事？

无疑地 `Student` 的 *copy* 构造函数会被调用，以 `plato` 为蓝本将 `s` 初始化。同样明显地，当 `validateStudent` 返回 `s` 会被销毁。因此，对此函数而言，参数的传递成本是“一次 `Student` *copy* 构造函数调用，加上一次 `Student` 析构函数调用”。

但那还不是整个故事喔。`Student` 对象内有两个 `string` 对象，所以每次构造一个 `Student` 对象也就构造了两个 `string` 对象。此外 `Student` 对象继承自 `Person` 对象，所以每次构造 `Student` 对象也必须构造出一个 `Person` 对象。一个 `Person` 对象又有两个 `string` 对象在其中，因此每一次 `Person` 构造动作又需承担两个 `string` 构造动作。最终结果是，以 *by value* 方式传递一个 `Student` 对象会导致调用一次 `Student` *copy* 构造函数、一次 `Person` *copy* 构造函数、四次 `string` *copy* 构造函数。当函数内的那个 `Student` 复件被销毁，每一个构造函数调用动作都需要一个对应的析构函数调用动作。因此，以 *by value* 方式传递一个 `Student` 对象，总体成本是“六次构造函数和六次析构函数”！

这是正确且值得拥有的行为，毕竟你希望你的所有对象都能够被确实地构造和析构。但尽管如此，如果有什么方法可以回避所有那些构造和析构动作就太好了。有的，就是 *pass by reference-to-const*：

```
bool validateStudent(const Student& s);
```

这种传递方式的效率高得多：没有任何构造函数或析构函数被调用，因为没有任何新对象被创建。修订后的这个参数声明中的 `const` 是重要的。原先的 `validateStudent` 以 *by value* 方式接受一个 `Student` 参数，因此调用者知道他们受到保护，函数内绝不会对传入的 `Student` 作任何改变；`validateStudent` 只能够对其复件（副本）做修改。现在 `Student` 以 *by reference* 方式传递，将它声明为 `const` 是必要的，因为不这样做的话调用者会忧虑 `validateStudent` 会不会改变他们传入的那个 `Student`。

以 *by reference* 方式传递参数也可以避免 *slicing*（对象切割）问题。当一个 `derived class` 对象以 *by value* 方式传递并被视为一个 `base class` 对象，`base class` 的 *copy* 构造函数

数会被调用，而“造成此对象的行为像个 **derived class** 对象”的那些特化性质全被切割掉了，仅仅留下一个 **base class** 对象。这实在不怎么让人惊讶，因为正是 **base class** 构造函数建立了它。但这几乎绝不会是你想要的。假设你在一组 **classes** 上工作，用来实现一个图形窗口系统：

```
class Window {
public:
    ...
    std::string name() const;           //返回窗口名称
    virtual void display() const;      //显示窗口和其内容
};

class WindowWithScrollBars: public Window {
public:
    ...
    virtual void display() const;
};
```

所有 **Window** 对象都带有一个名称，你可以通过 **name** 函数取得它。所有窗口都可显示，你可以通过 **display** 函数完成它。**display** 是个 **virtual** 函数，这意味简易朴素的 **base class** **Window** 对象的显示方式和华丽高贵的 **WindowWithScrollBars** 对象的显示方式不同（见条款 34 和条款 36）。

现在假设你希望写个函数打印窗口名称，然后显示该窗口。下面是错误示范：

```
void printNameAndDisplay(Window w)           //不正确! 参数可能被切割。
{
    std::cout << w.name();
    w.display();
}
```

当你调用上述函数并交给它一个 **WindowWithScrollBars** 对象，会发生什么事呢？

```
WindowWithScrollBars wwsb;
printNameAndDisplay(wwsb);
```

喔，参数 **w** 会被构造成为一个 **Window** 对象；它是 **passed by value**，还记得吗？而造成 **wwsb** “之所以是个 **WindowWithScrollBars** 对象”的所有特化信息都会被切除。在 **printNameAndDisplay** 函数内不论传递过来的对象原本是什么类型，参数 **w** 就像一个 **Window** 对象（因为其类型是 **Window**）。因此在 **printNameAndDisplay** 内调用 **display** 调用的总是 **Window::display**，绝不会是 **WindowWithScrollBars::display**。

解决切割 (slicing) 问题的办法, 就是以 *by reference-to-const* 的方式传递 *w*:

```
void printNameAndDisplay(const Window& w)           //很好,参数不会被切割
{
    std::cout << w.name();
    w.display();
}
```

现在, 传进来的窗口是什么类型, *w* 就表现出那种类型。

如果窥视 C++ 编译器的底层, 你会发现, *references* 往往以指针实现出来, 因此 *pass by reference* 通常意味真正传递的是指针。因此如果你有个对象属于内置类型 (例如 *int*), *pass by value* 往往比 *pass by reference* 的效率高些。对内置类型而言, 当你有机会选择采用 *pass-by-value* 或 *pass-by-reference-to-const* 时, 选择 *pass-by-value* 并非没有道理。这个忠告也适用于 STL 的迭代器和函数对象, 因为习惯上它们都被设计为 *passed by value*。迭代器和函数对象的实践者有责任看看它们是否高效且不受切割问题 (slicing problem) 的影响。这是“规则之改变取决于你使用哪一部分 C++ (见条款 1)”的一个例子。

内置类型都相当小, 因此有人认为, 所有小型 *types* 都是 *pass-by-value* 的合格候选人, 甚至它们是用用户自定义的 *class* 亦然。这是个不可靠的推论。对象小并不就意味着其 *copy* 构造函数不昂贵。许多对象——包括大多数 STL 容器——内含的东西只比一个指针多一些, 但复制这种对象却需承担“复制那些指针所指的每一样东西”。那将非常昂贵。

即使小型对象拥有并不昂贵的 *copy* 构造函数, 还是可能有效率上的争议。某些编译器对待“内置类型”和“用户自定义类型”的态度截然不同, 纵使两者拥有相同的底层表述 (underlying representation)。举个例子, 某些编译器拒绝把只由一个 *double* 组成的对象放进缓存器内, 却很乐意在一个正规基础上对光秃秃的 *doubles* 那么做。当这种事发生, 你更应该以 *by reference* 方式传递此等对象, 因为编译器当然会将指针 (*references* 的实现体) 放进缓存器内, 绝无问题。

“小型的用户自定义类型不必然成为 *pass-by-value* 优良候选人”的另一个理由是, 作为一个用户自定义类型, 其大小容易有所变化。一个 *type* 目前虽然小, 将来也许会变大, 因为其内部实现可能改变。甚至当你改用另一个 C++ 编译器都有可能改变 *type* 的大小。举个例子, 在我下笔此刻, 某些标准程序库实现版本中的 *string* 类型比其他版本大七倍。

一般而言，你可以合理假设“*pass-by-value* 并不昂贵”的唯一对象就是内置类型和 STL 的迭代器和函数对象。至于其他任何东西都请遵守本条款的忠告，尽量以 *pass-by-reference-to-const* 替换 *pass-by-value*。

请记住

- 尽量以 *pass-by-reference-to-const* 替换 *pass-by-value*。前者通常比较高效，并可避免切割问题 (slicing problem)。
- 以上规则并不适用于内置类型，以及 STL 的迭代器和函数对象。对它们而言，*pass-by-value* 往往比较适当。

条款 21：必须返回对象时，别妄想返回其 reference

Don't try to return a reference when you must return an object.

一旦程序员领悟了 *pass-by-value* (传值) 的效率牵连层面 (见条款 20)，往往变成十字军战士，一心一意根除 *pass-by-value* 带来的种种邪恶。在坚定追求 *pass-by-reference* 的纯度中，他们一定会犯下一个致命错误：开始传递一些 *references* 指向其实并不存在的对象。这可不是件好事。

考虑一个用以表现有理数 (rational numbers) 的 class，内含一个函数用来计算两个有理数的乘积：

```
class Rational {
public:
    Rational(int numerator = 0,          //条款 24 说明为什么这个构造函数
             int denominator = 1);      //不声明为 explicit
    ...
private:
    int n, d;                          //分子 (numerator) 和分母 (denominator)
    friend
        const Rational
            operator* (const Rational& lhs,
                       const Rational& rhs);
};
```

这个版本的 *operator** 系以 *by value* 方式返回其计算结果 (一个对象)。如果你完全不担心该对象的构造和析构成本，你其实是明显逃避了你的专业责任。若非必要，没有人会想要为这样的对象付出太多代价，问题是需要付出任何代价吗？

唔, 如果可以改而传递 reference, 就不需付出代价。但是记住, 所谓 reference 只是个名称, 代表某个既有对象。任何时候看到一个 reference 声明式, 你都应该立刻问自己, 它的另一个名称是什么? 因为它一定是某物的另一个名称。以上述 operator* 为例, 如果它返回一个 reference, 后者一定指向某个既有的 Rational 对象, 内含两个 Rational 对象的乘积。

我们当然不可能期望这样一个 (内含乘积的) Rational 对象在调用 operator* 之前就存在。也就是说, 如果你有:

```
Rational a(1, 2);           //a = 1/2
Rational b(3, 5);           //b = 3/5
Rational c = a * b;         //c 应该是 3/10
```

期望“原本就存在一个其值为 3/10 的 Rational 对象”并不合理。如果 operator* 要返回一个 reference 指向如此数值, 它必须自己创建那个 Rational 对象。

函数创建新对象的途径有二: 在 stack 空间或在 heap 空间创建之。如果定义一个 local 变量, 就是在 stack 空间创建对象。根据这个策略试写 operator* 如下:

```
const Rational& operator* (const Rational& lhs,
                           const Rational& rhs)
{
    Rational result(lhs.n * rhs.n, lhs.d * rhs.d);    //警告! 糟糕的代码!
    return result;
}
```

你可以拒绝这种做法, 因为你的目标是要避免调用构造函数, 而 result 却必须像任何对象一样地由构造函数构造起来。更严重的是: 这个函数返回一个 reference 指向 result, 但 result 是个 local 对象, 而 local 对象在函数退出前被销毁了。因此, 这个版本的 operator* 并未返回 reference 指向某个 Rational, 它返回的 reference 指向一个“从前的”Rational; 一个旧时的 Rational; 一个曾经被当做 Rational 但如今已经成空、发臭、败坏的残骸, 因为它已经被销毁了。任何调用者甚至只是对此函数的返回值做任何一点点运用, 都将立刻坠入“无定义行为”的恶地。事情的真相是, 任何函数如果返回一个 reference 指向某个 local 对象, 都将一败涂地。(如果函数返回指针指向一个 local 对象, 也是一样。)

于是，让我们考虑在 heap 内构造一个对象，并返回 reference 指向它。Heap-based 对象由 new 创建，所以你得写一个 heap-based operator* 如下：

```
const Rational& operator* (const Rational& lhs,
                          const Rational& rhs)
{
    Rational* result = new Rational(lhs.n * rhs.n, lhs.d * rhs.d);
    return *result;
}
```

唔，你还是必须付出一个“构造函数调用”代价，因为分配所得的内存将以一个适当的构造函数完成初始化动作。但此外你现在又有了另一个问题：谁该对着被你 new 出来的对象实施 delete？

即使调用者诚实谨慎，并且出于良好意识，他们还是不太能够在这样合情合理的用法下阻止内存泄漏：

```
Rational w, x, y, z;
w = x * y * z;           //与 operator*(operator*(x, y), z) 相同
```

这里，同一个语句内调用了两次 operator*，因而两次使用 new，也就需要两次 delete。但却没有合理的办法让 operator* 使用者进行那些 delete 调用，因为没有合理的办法让他们取得 operator* 返回的 references 背后隐藏的那个指针。这绝对导致资源泄漏。

但或许你注意到了，上述不论 on-the-stack 或 on-the-heap 做法，都因为对 operator* 返回的结果调用构造函数而受惩罚。也许你还记得我们的最初目标是要避免如此的构造函数调用动作。或许你认为你知道有一种办法可以避免任何构造函数被调用。或许你心里出现下面这样的实现代码，此法莫基于“让 operator* 返回的 reference 指向一个被定义于函数内部的 static Rational 对象”：

```
const Rational& operator* (const Rational& lhs,
                          const Rational& rhs)
{
    static Rational result;           //警告，又一堆烂代码。
    //static 对象，此函数将返回
    // 其reference。
    result = ... ;                   //将 lhs 乘以 rhs，并将结果
    // 置于 result 内。
    return result;
}
```

就像所有用上 `static` 对象的设计一样, 这一个也立刻造成我们对多线程安全性的疑虑。不过那还只是它显而易见的弱点。如果想看看更深层的瑕疵, 考虑以下面这些完全合理的客户代码:

```
bool operator==(const Rational& lhs,          //一个针对 Rationals
                const Rational& rhs); // 而写的 operator==
Rational a, b, c, d;
...
if ((a * b) == (c * d)) {
    当乘积相等时, 做适当的相应动作;
} else {
    当乘积不等时, 做适当的相应动作;
}
```

猜想怎么着? 表达式 `((a*b) == (c*d))` 总是被核算为 `true`, 不论 `a, b, c` 和 `d` 的值是什么!

一旦将代码重新写为等价的函数形式, 很容易就可以了解出了什么意外:

```
if (operator==(operator*(a, b), operator*(c, d)))
```

注意, 在 `operator==` 被调用前, 已有两个 `operator*` 调用式起作用, 每一个都返回 `reference` 指向 `operator*` 内部定义的 `static Rational` 对象。因此 `operator==` 被要求将 “`operator*` 内的 `static Rational` 对象值” 拿来和 “`operator*` 内的 `static Rational` 对象值” 比较, 如果比较结果不相等, 那才奇怪呢。(译注: 这里我补充说明: 两次 `operator*` 调用的确各自改变了 `static Rational` 对象值, 但由于它们返回的都是 `reference`, 因此调用端看到的永远是 `static Rational` 对象的 “现值”。)

这应该足够说服你, 欲令诸如 `operator*` 这样的函数返回 `reference`, 只是浪费时间而已, 但现在或许又有些人这样想: “唔, 如果一个 `static` 不够, 或许一个 `static array` 可以得分……”。

我不打算再次写出示例来驳斥这个想法以彰显自己多么厉害, 但我可以简单描述为什么你该为了提出这个念头而脸红。首先你必须选择 `array` 大小 `n`。如果 `n` 太小, 你可能会耗尽 “用以存储函数返回值” 的空间, 那么情况就回到了我们刚才讨论过的单一 `static` 设计。但如果 `n` 太大, 会因此降低程序效率, 因为 `array` 内的每一个对象都会在函数第一次被调用时构造完成。那么将消耗 `n` 个构造函数和 `n` 个析构函数——即使我们所讨论的函数只被调用一次。如果所谓 “最优化” 是改善软件效率的过程, 我们现在所谈的这些应该称为 “恶劣化”。最后, 想一想如何将你需要的值放进 `array` 内,